Muhammad Ibtisam Iqbal

DevOps & Cloud Engineer

contact@ibtisam-iq.com | +92 304 6210233 | Islamabad, Pakistan | linkedin.com/in/ibtisam-iq | github.com/ibtisam-iq | ibtisam-iq.com

PROFESSIONAL SUMMARY

CKA and CKAD certified DevOps & Cloud Engineer with hands-on experience building Kubernetes clusters, CI/CD pipelines, and AWS infrastructure from first principles. Audited and modernized application codebases across Java, Python, and Node.js (dependency upgrades, config externalization, health endpoints, architectural refactoring) before containerizing and deploying them. Built and documented 8 infrastructure projects end-to-end covering microservices, monoliths, and cloud-native platforms, each with source code, runbooks, and terminal sessions. Looking for a role where I can bring the same ownership to production systems and grow within a team.

CERTIFICATIONS

Certified Kubernetes Administrator (CKA)
Cloud Native Computing Foundation (CNCF) · Nov 2025 · Valid until Nov 2027

Certified Kubernetes Application Developer (CKAD)
Cloud Native Computing Foundation (CNCF) · Nov 2025 · Valid until Nov 2027

Certified Kubernetes Security Specialist (CKS)
In Progress · Cloud Native Computing Foundation (CNCF)

TECHNICAL SKILLS

Containers & Orchestration: Kubernetes, kubeadm, Docker, Docker Buildx, Docker Compose, Helm, Helmfile, Kustomize, ArgoCD, ArgoCD Image Updater

Cloud (AWS): EKS, ECS, EC2, Auto Scaling, S3, RDS, DynamoDB, VPC, NAT Gateway, VPC Peering, IAM, Route 53, CloudFront, KMS, CloudFormation, SQS, Lambda, SNS, ACM, CloudTrail

CI/CD & DevSecOps: Git, GitHub Actions, Jenkins, SonarQube, Nexus, Trivy, Hadolint, Bandit, pip-audit, Maven, Make

Infrastructure as Code: Terraform, Ansible, Bash, AWS CLI, eksctl

Networking: Gateway API, Ingress, AWS Load Balancer Controller, Nginx, ExternalDNS, Cloudflare Tunnel, DNS, TLS/SSL, OpenSSL

Observability: Prometheus, Grafana, AlertManager, Elasticsearch, Kibana, Filebeat, Fluent Bit, CloudWatch

Linux & Databases: Ubuntu, Alpine Linux, systemd, PostgreSQL, MySQL, Redis, RabbitMQ

Practices: GitOps, DevSecOps, Application Modernization, Microservices Architecture, Cloud Native, Infrastructure Automation, Security & Compliance, Load Balancing, Blue/Green & Canary Deployments, Virtualization, OS Hardening, Disaster Recovery, Troubleshooting, 12-Factor App

PROJECTS

Microservices GitOps on Amazon EKS — *Terraform, EKS, ArgoCD, Gateway API, Prometheus, Elastic Stack*

- Deployed 10 microservices on Amazon EKS using modular Terraform (VPC, bastion, self-managed ASGs) and 3 GitHub Actions CI pipelines with dynamic matrix dispatch.
- Automated GitOps delivery: CI pushes images to GHCR, ArgoCD Image Updater detects new SHA-256 digests and triggers EKS rollouts with zero manual intervention.
- Built monorepo CI with per-service change detection: compares Git history to identify modified services, fans out parallel builds only for affected applications, and isolates failures so one broken service does not block others.
- Kept the upstream Helm chart unmodified. Used Kustomize within ArgoCD to inject custom Gateway API manifests as overlays.
- Configured Gateway API routing with a shared ALB across 5 HTTPRoutes, ExternalDNS for automatic Route 53 record creation, and HPA for CPU-based autoscaling via Metrics Server.
- Provisioned an ACM wildcard certificate for TLS across all subdomains. Configured the EBS CSI driver with gp3 StorageClass for Elasticsearch PVCs.
- Deployed observability independently of the application: kube-prometheus-stack for metrics (Slack alerts via AlertManager) and Elastic Stack for centralized logging across a 4-node cluster.

SilverStack: Self-Hosted CI/CD Platform *systemd, Docker, Jenkins, SonarQube, Nexus, Cloudflare Tunnel*

- Built a self-hosted CI/CD platform (Jenkins, SonarQube, Nexus) on microVMs with 5 custom OCI rootfs images, each using systemd as PID 1 for runtime provisioning at boot.
- Configured Nginx as a reverse proxy in each image with a build-time parameterized upstream port, /health endpoint for readiness checks, and hardened security headers (X-Frame-Options, X-Content-Type-Options, Referrer-Policy). All services serve on port 80.
- Configured outbound-only Cloudflare Tunnels to expose services on custom domains without public IPs, bypassing Carrier-Grade NAT (CGNAT).
- Automated image CI with GitHub Actions: build-time health checks validate systemd units, Nginx configs, and service readiness before publishing to GHCR.
- Enforced security through strict sudoers profiles, daemon user isolation, pinned binary versions, and SHA256 checksum verification on all downloads.
- Each image is independently launchable (standalone SonarQube, Nexus, Dev Machine, or full CI/CD stack) and published as a publicly accessible playground on iximiuz Labs.

14-Stage DevSecOps Pipeline *Jenkins, GitHub Actions, Trivy, SonarQube, GHCR, Docker Hub, Nexus*

- Validated each application's build and test cycle locally (Maven, Webpack, pytest with real databases) before writing any pipeline automation.
- Built a standardized 14-stage CI pipeline for 3 codebases (Java, Python, Node.js) and implemented it identically on Jenkins and GitHub Actions to prove tool-agnostic design.
- Integrated language-specific security scanning: Bandit and pip-audit for Python, npm audit and ESLint for Node.js, Trivy filesystem scan for Java.
- Configured 3-pass Trivy container scanning: library vulnerabilities hard-fail the build on CRITICAL findings, OS vulnerabilities run as advisory-only to prevent false blocks from upstream CVEs.
- Tagged every image with semantic version, 7-character Git SHA, and CI build number. No floating 'latest' tags. Published to GHCR, Docker Hub, and self-hosted Nexus simultaneously.
- Ran CI test suites against in-memory databases (SQLite for Python) to avoid external dependencies, while still generating JaCoCo and Cobertura XML coverage reports for SonarQube quality gates.
- Enforced strict CI/CD separation: pipelines commit image SHAs to the CD repository. No pipeline has kubectl access to any cluster.

Multi-Environment Deployment: Bare-Metal to Amazon EKS *Helmfile, kubeadm, EKS, IRSA, DynamoDB, SQS, Lambda, SNS*

- Deployed 5 microservices across bare-metal Kubernetes (kubeadm) and Amazon EKS. Kept all upstream Helm charts unmodified and decoupled AWS dependencies by swapping DynamoDB, SQS, and ElastiCache for containerized alternatives (RabbitMQ, Redis) on bare metal.
- Wrote 3 Helmfile configurations with dependency sequencing to prevent startup race conditions (databases and brokers provision before application services).
- Created self-managed CloudFormation worker nodes to bypass KodeKloud lab SCP restrictions on EKS managed node groups. Configured IRSA to bind services to DynamoDB and a serverless event pipeline (SQS, Lambda, SNS).
- Deployed Fluent Bit to stream logs to CloudWatch Container Insights. Set up an ALB Ingress Group to share a single load balancer across the UI, Prometheus, and Grafana.

Polyglot Application Modernization Across 4 Compute Models *Java, Python, Node.js, Docker, Kustomize, kubeadm, EKS, ECS, EC2*

- Audited and modernized 3 existing codebases (Java Spring Boot 3.4, Python Flask, Node.js Express): upgraded frameworks, pinned dependencies, externalized all config to environment variables, and injected container health endpoints.
- Refactored the Node.js app from 2-tier to 3-tier architecture (Nginx serving React, proxying API to Express). Built multi-stage Dockerfiles for each stack. Validated each build on bare metal before containerizing.
- Debugged environment-specific edge cases across all 3 stacks: python-dotenv URL truncation, PostgreSQL 15+ schema-level grants, and JDBC URL quoting in shell scripts.
- Kept the Python Flask app as a single-container deployment with Gunicorn instead of forcing a microservice split. Applied the architecture that matched the application, not the trend.
- Deployed all 3 applications across EC2, ECS Fargate, bare-metal Kubernetes (kubeadm), and Amazon EKS. Wrote Kustomize overlays sharing 80% of base manifests between environments.

BankApp: Same Artifact Across EC2, ECS, and EKS *EC2, ECS Fargate, EKS, RDS, Gateway API, Kustomize, Docker*

- Deployed the same Java artifact (Spring Boot 3.4, Java 21) across EC2 Auto Scaling, ECS Fargate, and Amazon EKS to compare each compute model firsthand. All three share one VPC and RDS instance.
- Decoupled the embedded database to Amazon RDS, reduced container image by 60% with multi-stage builds, and aligned JVM memory parameters with Kubernetes resource limits to prevent OOM kills.

DebugBox: Open-Source Kubernetes Diagnostics Toolkit *Alpine Linux, Docker Buildx, Trivy, Hadolint, GitHub Actions, MkDocs*

- Built an open-source Kubernetes debugging toolkit with 3 Alpine-based variants (15MB to 91MB). The smallest variant is 93% smaller than netshoot.
- Separated Dockerfiles (declarative) from installation scripts (procedural). Package lists live in '.packages' files; complex installs use dedicated scripts with architecture detection and SHA256 verification.
- Pre-loaded kubectx/kubens and custom bash functions (json(), yaml(), sniff-dns(), cert-check()) for faster debugging sessions.
- Automated multi-arch builds (amd64, arm64) with Trivy gating on HIGH/CRITICAL findings, Hadolint linting, in-container smoke tests, and dual-registry publishing (GHCR + Docker Hub).
- Published documentation via MkDocs and GitHub Pages. Wrote CONTRIBUTING.md, CODE_OF_CONDUCT.md, and SECURITY.md for open-source governance.

AWS Static Hosting with CloudFront OAC and Cross-Region Replication *S3, CloudFront, KMS, Route 53, CloudTrail, IAM*

- Built a zero-compute hosting architecture on AWS using CloudFront with Origin Access Control (OAC/SigV4) against a fully private S3 origin. Configured Custom Error Responses to handle SPA client-side routing.
- Configured Cross-Region Replication (us-east-1 to us-west-2) with separate KMS Customer Managed Keys per region. Enabled S3 Bucket Keys to reduce KMS API costs.
- Set up CloudTrail for management and data event auditing across both regions. Enabled S3 Server Access Logs for HTTP telemetry. Configured Glacier lifecycle policies for versioned content.
- Deployed on a personal AWS account and kept live for 7 days. Total cloud cost: $1.45 across CloudFront, S3 (3 buckets), CloudTrail, and KMS, covered by AWS Free Tier credits.

EXPERIENCE

DevOps Engineer · *Independent Engineering Projects (Self-Driven)* Apr 2024 – Present

- Covered the full DevOps lifecycle across 3 language ecosystems (Java, Python, Node.js): audited codebases, wrote Dockerfiles, built CI/CD pipelines on Jenkins and GitHub Actions, deployed across 4 compute models (EC2, ECS, kubeadm, EKS), and implemented GitOps delivery with ArgoCD (detailed in Projects section).
- Earned CKA and CKAD certifications (both performance-based, 2-hour exams in live Kubernetes clusters). Published certification prep (domain breakdowns, practice scenarios) to cert-vault.ibtisam-iq.com, and documented all projects with runbooks, terminal sessions, and source code.

Virtual Assistant · *Independent Clients (Amazon FBA Private Label) · Freelance* Apr 2022 – Apr 2024

- Managed Amazon FBA Private Label operations remotely for international clients: coordinated suppliers, logistics, inventory, and performance tracking end-to-end.
- Self-funded the transition into DevOps engineering through freelancing income. Built workflow discipline, process thinking, and remote collaboration skills.

EDUCATION & TRAINING

AWS Solutions Architect Training (100 hrs) + Linux Administration (60 hrs) · *Technical Guftgu* Nov 2025

DevSecOps & Cloud DevOps BootCamp · *DevOps Shack* Jan 2025

Hands-on Lab Platforms · *KodeKloud (CKA/CKAD Prep, AWS Labs) + iximiuz Labs (5 Published Playground Images)* 2024 – Present

M.Sc. Horticulture · *University of Agriculture, Faisalabad* (Silver Medal, Mar 2024) 2022

PORTFOLIO & DOCUMENTATION

Projects: projects.ibtisam-iq.com
Cert Vault: cert-vault.ibtisam-iq.com
Blog: blog.ibtisam-iq.com
SilverStack: github.com/ibtisam-iq/silver-stack
Runbook: runbook.ibtisam-iq.com
Knowledge Base: nectar.ibtisam-iq.com
DebugBox: debugbox.ibtisam-iq.com
iximiuz Labs: labs.iximiuz.com/a/ibtisam-iq